Benchmarking Superconducting Qubits with Interleaved Randomized Benchmarking

Laura L. Vilela, Faculty of Science and Technology, University of Coimbra

***Abstract*— This project investigates the fidelity of quantum gates in superconducting qubits using Randomised Benchmarking (RB) and Interleaved Randomised Benchmarking (IRB). Experiments will be implemented in Qiskit and executed both on noisy simulators with device-calibrated noise models and on real IBM Quantum hardware. The study aims to extract the error per Clifford (EPC) for single- and two-qubit gates, analyse discrepancies between simulation and hardware, and relate these errors to physical noise mechanisms such as decoherence times (T_1, T_2) and readout errors. The results will provide quantitative insight into hardware limitations and contribute to the understanding of gate performance in near-term quantum devices, while demonstrating practical implementation of quantum benchmarking protocols.**

I. Introduction

Superconducting qubits are among the leading platforms for near-term quantum computing, offering fast gate operations and scalability potential. However, their performance is limited by decoherence, control errors, and readout imperfections. Quantifying the fidelity of quantum gates is therefore essential to understand hardware limitations and to guide optimisation strategies.

Randomised Benchmarking (RB) is a widely used protocol to measure the average error per Clifford gate in a scalable and robust manner, mitigating the effects of state preparation and measurement errors. Interleaved Randomised Benchmarking (IRB) extends this approach to estimate the fidelity of specific gates.

In this project, we implement RB and IRB using Qiskit and Qiskit Experiments, executing experiments both in simulators with noise models extracted from IBM Quantum devices and on real superconducting hardware. This approach allows us to compare idealised noise models with actual hardware performance, relate errors to physical mechanisms such as T_1, T_2, and readout noise, and provide insights into the reliability of quantum operations in near-term devices.

There are various hardware platforms for qubit implementation. As of 2025, the leading technologies for quantum processors, in terms of scalability, use neutral atoms in optical lattices (Atom Computing, >1200 qubits [1]), superconducting qubits (IBM, Condor, 1121 qubits [2]), and photonics measured-based (Xanadu, Borealis, 216 qubits [3]). However, in terms of fidelity, trapped ion technologies seem to be leading the industry (Quantinuum, H2, 99.997% - 1 qubit, 99.87% - 2 qubit [4]).

A. Scalability vs Fidelity

While scalability is a classical concept, it gains new dimension when used in the context of quantum computing. Because qubits rely on nanoscale phenomena, they are highly susceptible to interference, both from environmental noise and neighbouring qubits. For this reason, quantifying the maximum number of qubits that can operate simultaneously without performance loss is a crucial factor in system design [5].

Fidelity characterizes the agreement between the actual outcome of the operation and the desired state. It generally depends on the initial state on which the operation is applied. Since the operation may generally be applied to any arbitrary state, it is natural to average the fidelity over all pure initial states, chosen uniformly in the system's Hilbert space. [6].

II. Interleaved Randomised Benchmarking

A. Interleaved Randomised Benchmarking

Randomised Benchmarking proves to be advantageous over different qubit characterisation techniques, such as Quantum Process Tomography. While RB is scalable in terms of number of qubits comprising the system, it is also able to provide a noise measure independent of state preparation and measurement errors (SPME) [7].

The key difference between RB and IRB, is that the first characterises average impact of noise in gates, estimating an average error per Clifford.The Clifford group can be defined as the group of operators $\mathcal{C}$ that normalises the Pauli group $\mathcal{P}$, that is: for each$C \in \mathcal{C}$ and $P \in \mathcal{P}$, $CPC^\dagger \in \mathcal{P}$ On the other hand, IRB characterises a particular gate, which is 'interleaved' throughout the standard RB sequence. This protocol extends standard RB by interleaving the gate of interest after every Clifford gate. We can see the difference between the sequences quite clearly: [8].

$$RB_{\text{seq}} = C_1, C_2, C_3, \ldots, C_{\text{N-1}}, C'_{\text{inverse}}$$
$$IRB_{\text{seq}} = C_1, G, C_2, G, C_3, \ldots, C_{\text{N-1}}, G, C'_{\text{inverse}}$$

B. Standardised Protocol

The protocol that will be followed consists of 3 steps. After the initial state is prepared, the standard RB will be applied, followed by the procedures for survival probability estimation. Finally, gate error will be calculated.

0) Fixing the initial state $|\psi\rangle$
   The initial state will be set to $|\psi\rangle = |0\rangle$, as standard.

1) Standard RB implementation
   A K sequence of length m is defined by selecting m gates, chosen uniformly at random, from Clif_n. The $(m+i)$-th gate shall be the inverse of the composition

of the first m gates, to make the total sequence equal to the identity operation. Each Clifford element, C_i, has associated to it an error Λ_i. The sequence of gates can then be represented as:

$$\mathbf{S}_{im} = \Lambda_{i_{m+1},m+1} \otimes \mathcal{C}_{i_{m+1}} \otimes \left(\bigotimes_{j=1}^{m} \left[\Lambda_{i_j,j} \otimes \mathcal{C}_{i_j} \right] \right) \tag{1}$$

For each K sequence, survival probability, p is measured. This is the probability that the initial state is not changed by the sequence.

$$S_p = Tr[E_\psi \mathcal{S}_{i_m}(\rho_\psi)] \tag{2}$$

Here E_ψ is the positive operator valued measurement element that takes into account measurement errors and (ρ_ψ) is a quantum state that takes into account state preparation errors. The survival probability shall be averaged over the K random sequences in order to find the sequence fidelity, $F_{\text{seq}}(m, \psi)$. This value is then fit to either the zeroth or first order model:

$$\begin{aligned} F^{(0)}_{\text{seq}}(m, \psi) &= A_0 p^m + B_0, \\ F^{(1)}_{\text{seq}}(m, \psi) &= A_1 p^m + C_1(m-1)p^{m-2} + B_1, \end{aligned} \tag{3}$$

This fit gives the depolarising parameter, p. This allows to calculate the average error gate over all Clifford gates, r.

$$r = (d-1)(1-p)/d \tag{4}$$

2) Interleaving an arbitrary gate to the RB sequence
The K sequences shall be altered so each random Clifford gate, C_i, is followed by the target gate, C. This way, the sequence is constructed by alternating between uniformly random Clifford elements and the target gate up to the m-th random gate. Similarly to RB, the $(m+1)$-th fate is chosen to be the inverse of the composition that precedes it. The superoperator that represents this sequence is:

$$\mathcal{V}_{i_m} = \Lambda_{i_{m+1},m+1} \otimes \mathcal{C}_{i_{m+1}} \otimes \left(\bigotimes_{j=1}^{m} \left[\mathcal{C} \otimes \Lambda_C \otimes \Lambda_{i_j,j} \otimes \mathcal{C}_{i_j} \right] \right) \tag{5}$$

3) Gate Error estimation Analogously to standardised RB, For each K sequence, survival probability, p is measured.

$$S_p = Tr[E_\psi \mathcal{V}_{i_m}(\rho_\psi)] \tag{6}$$

By averaging this value over all K sequences, we can find the new gate fidelity, $F_{\overline{\text{seq}}}(m, \psi)$. By using equation (3), fitting $F_{\overline{\text{seq}}}(m, \psi)$ allows us to calculate the depolarising parameter $p_{\overline{c}}$, where $F_{\overline{\text{seq}}}(m, \psi)$ substitutes p in the expressions.
The gate error of Λ_C (which is directly is estimated by ($r_{\mathcal{C}} = 1 -$ average gate fidelity of $\Lambda_{\mathcal{C}}$) is calculated from the values obtained for p and $p_{\overline{c}}$:

$$r_c^{\text{est}} = \frac{(d-1)\left(1 - \frac{p_{\overline{c}}}{p}\right)}{d} \pm E. \tag{7}$$

Here E is the error in our estimate and is given by:

$$E = \min \begin{cases} \dfrac{(d-1)\left|p - \frac{p_{\overline{c}}}{p}\right| + (1-p)}{d} \\ \dfrac{2(d^2-1)(1-p)}{pd^2} + \dfrac{4\sqrt{1-p}\sqrt{d^2-1}}{p} \end{cases} \tag{8}$$

C. Two-Qubit Protocol

As can be seen in Ref [**?**] The two-qubit protocol uses the same mathematical formulas to fit, obtain parameters p and $p_{\overline{c}}$ and calculate r and r_C. The main difference lies in how the sequences are defined.

1) RB Sequence Similarly, a sequence K of length m is constructed by selecting gates from Clif_n at random. From this sequence, a series of truncations t_i is constructed, where $t_i = \{C_1, ..., C_N\}$. To make each truncation self-inverting, a final pulse I_i is then appended at the eand of each truncation. This pulse returns the system to the $|00\rangle$ state. The resulting RB sequence consists in a series of pulse trains: $K_i = \{t_i, I_i\}$
2) IRB sequence Like in the single-qubit case, the gate we wish to characterise is interleaved between Clifford gates chosen at random.

III. Simulation with Noisy Channels

In this section, we simulate quantum circuits using noise models that emulate real devices. For practical purposes, the IBM hardware platform chosen to be emulated were the circuits of the "ibm_fez" server. The server has the following properties:

Property	**Value**
Backend Name	`ibm_fez`
Qubits	156
Basis Gates	CZ, Id, RX, RZ, RZZ, SX, X
CZ error (median)	2.752×10^{-3}
SX error (median)	2.705×10^{-4}
Readout Error (median)	9.70×10^{-3}
T_1 (median)	$137.23\,\mu s$
T_2 (median)	$92.97\,\mu s$

TABLE I: Properties of the `ibm_fez` server.

The gate fidelities for gates X, SX, RX, RZ and CS were calculated using the protocol listed above. All code used to obtain the following results can be consulted in the annex.

A. Single-Qubit Gates

We will start by analysing the single-qubit gates X, SX, RX and RZ.

1) X Gate

The following table shows the fit parameters obtained from performing RB and IRB of Gate X:

	Parameters	$F_{\text{seq}}^{(0)}$	$F_{\text{seq}}^{(1)}$
RB	A	0.6974	0.4886
	B	0.2993	0.5076
	C	-	0.0001
	p	0.9999	0.999811
	r	5.000×10^{-5}	9.400×10^{-5}
IRB	A	0.698463	0.487385
	B	0.298272	0.508942
	C	-	0.000053
	$p_{\overline{c}}$	0.9999	0.99989679
	r_c	5.000×10^{-5}	5.160×10^{-5}

TABLE II: Simulation results for Gate X.

The results obtained for gate fidelities were $F_{seq} = \{0.996517, 0.995605, 0.996452, 0.995996, 0.996549\}$, for RB, and $F_{\overline{seq}} = \{0.996387, 0.995898, 0.996745, 0.995833, 0.996257\}$.The graphs below correspond to the data presented in Table II.

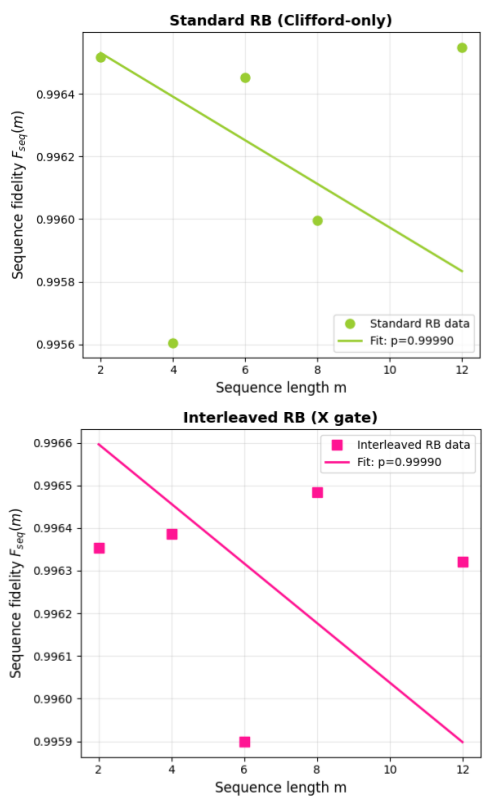

Fig. 1: Fidelity of X via Standard RB (top) and Interleaved RB (bottom)

For these parameters, the resulting gate fidelity was 100.00% for zero-th order and 100.00% for the first order. As we can see from the table, the gate errors r and r_c are extremely small, leading to very high fidelity. The similarity between RB and IRB results for the X gate means the X gate does not introduce significant additional error compared to the average Clifford gate. This is expected for a well-behaved single-qubit gate.

2) SX Gate

The following table presents the fit parameters obtained from RB of the gate SX. A first-order fit could not be performed in this case. However, since the SX gate is equivalent to an $RX(\pi/2)$ rotation, we expect the results to closely resemble those from the RX gate characterisation discussed below.

	Parameters	$F_{\text{seq}}^{(0)}$
RB	A	0.697551
	B	0.299712
	p	0.9999
	r	5.00×10^{-5}
IRB	A	0.698452
	B	0.298869
	$p_{\overline{c}}$	0.9999
	r_c	5.00×10^{-5}

TABLE III: Simulation results for Gate SX.

The results obtained for gate fidelities were $F_{seq} = \{0.996680, 0.997005, 0.996908, 0.996842, 0.996647\}$, for RB, and $F_{\overline{seq}} = \{0.997070, 0.996712, 0.997005, 0.996940, 0.996647\}$.The graphs below correspond to the data presented in Table III.

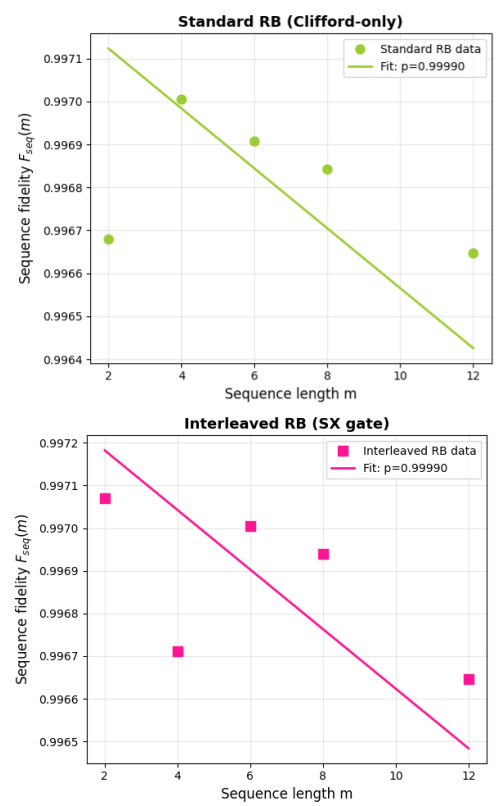

Fig. 2: Fidelity of SX via Standard RB (top) and Interleaved RB (bottom).

The obtained gate fidelity was 100.00% for RB. The first order fit was not possible to execute as the $F_{\overline{seq}}$ values are extremely close to each other and very close to 1.0, with almost no decay as m increases. This makes it very hard for a multi-parameter model to fit meaningfully, as there's not enough "signal" for the optimizer to distinguish between parameters, that is: the noise is negligible.

3) RX Gate
The following table shows the fit parameters obtained from performing RB and IRB of Gate RX:

	Parameters	$F_{\text{seq}}^{(0)}$	$F_{\text{seq}}^{(1)}$
RB	A	0.698991	0.487851
	B	0.298591	0.509355
	C	-	0.000091
	p	0.9999	0.99981987
	r	2.98591×10^{-1}	9.006×10^{-5}
IRB	A	0.698509	0.648785
	B	0.29878	0.352602
	C	-	0.005299
	$p_{\overline{c}}$	0.9999	0.99214165
	r_c	5.000×10^{-5}	3.8398×10^{-3}

TABLE IV: Simulation results for Gate RX.

The results obtained for gate fidelities were $F_{seq} = \{0.997038, 0.996615, 0.996745, 0.996940, 0.997331\}$, for RB, and $F_{\overline{seq}} = \{0.996712, 0.996615, 0.996842, 0.997493, 0.996549\}$.The graphs below correspond to the data presented in Table IV.

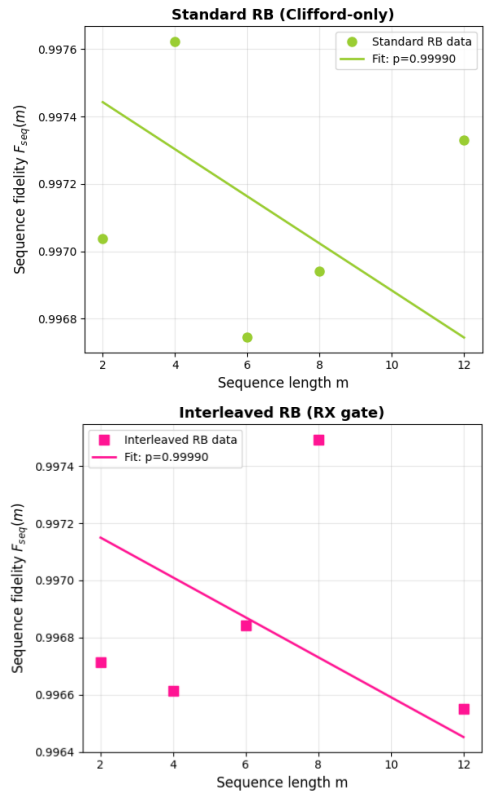

Fig. 3: Fidelity of RX via Standard RB (top) and Interleaved RB (bottom).

We have obtained the values 100.00%, for 0-th order fit and 99.61602% for 1-st order fit. It would be expectable that the SX gate would present similiar values.

4) RZ Gate
The following table shows the fit parameters obtained from performing RB and IRB of Gate RZ:

	Parameters	$F_{\text{seq}}^{(0)}$	$F_{\text{seq}}^{(1)}$
RB	A	0.697968	0.496473
	B	0.299549	0.502408
	C	-	0.001798
	p	0.9999	0.99639975
	r	5.000×10^{-5}	$1,80012 \times 10^{-3}$
IRB	A	0.698561	0.486923
	B	0.298963	0.510431
	C	-	0.000358
	$p_{\overline{c}}$	0.9999	0.99214165
	r_c	5.000×10^{-5}	$3,5388 \times 10^{-4}$

TABLE V: Simulation results for Gate RZ.

The results obtained for gate fidelities were $F_{seq} = \{0.997201, 0.997005, 0.996973, 0.997298, 0.996875\}$, for RB, and $F_{\overline{seq}} = \{0.99694, 0.997428, 0.996615, 0.997233, 0.997168\}$.The graphs below correspond to the data presented in Table V.

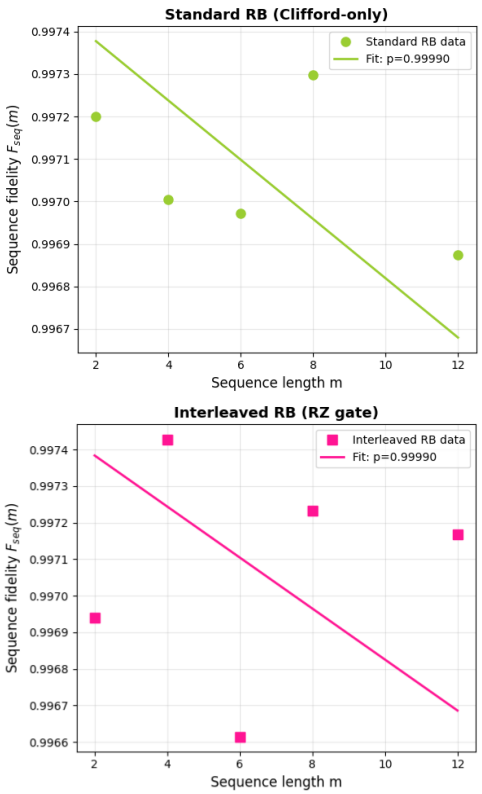

Fig. 4: Fidelity of RZ via Standard RB (top) and Interleaved RB (bottom).

The RZ gate fidelity values obtained were 100.00% for the 0-th order fit and 100% for 1-st order fit. These were the highest values obtained. One possible explanation is that since the RZ gate is not a real pulse, it does not introduce extra noise or error beyond what is already present in the system.

B. Two-Qubit Gates

In this section, the two-qubit gates will be testes using the adapted protocol. It is expected that, for superconducting platforms, they present higher error values.

1) CZ Gate We will start by characterising Gate CZ. This gate rotates the target qubit by an angle π around the z-axis of the Bloch sphere, changing its phase, if the control qubit is in state $|1\rangle$. The results obtained can be found in the table below.

	Parameters	$F_{\text{seq}}^{(0)}$	$F_{\text{seq}}^{(1)}$
RB	A	0.706472	0.794273
	B	0.28618	0.205322
	C	-	0.008337
	p	0.99988424	0.98988376
	r	8.682×10^{-5}	$5,05812 \times 10^{-3}$
IRB	A	0.693728	0.486923
	B	0.298643	0.510431
	C	-	0.000358
	$p_{\overline{c}}$	0.9999	0.90000001
	r_c	7.500×10^{-5}	$5,000 \times 10^{-2}$

TABLE VI: Simulation results for Gate CZ.

The results obtained for gate fidelities were $F_{seq} = \{0.991895, 0.992741, 0.992057, 0.992806, 0.991146\}$, for RB, and $F_{\overline{seq}} = \{0.990983, 0.992220, 0.992643, 0.992285, 0.991504\}$.The graphs below correspond to the data presented in Table VI.

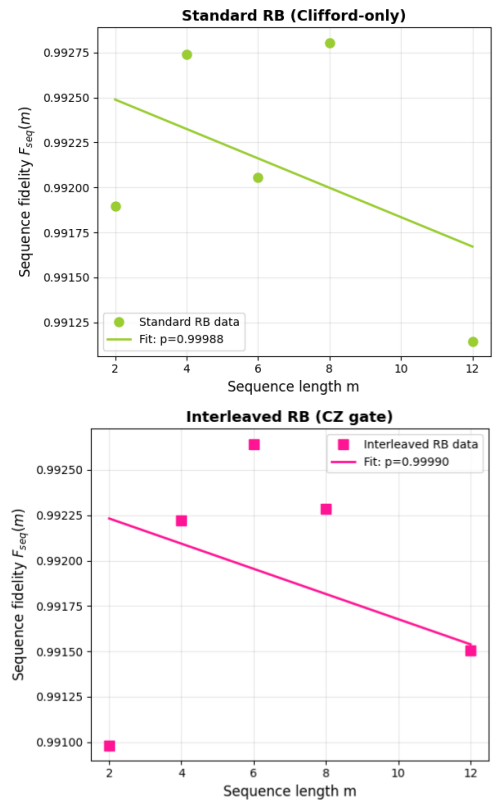

Fig. 5: Fidelity of CZ via Standard RB (top) and Interleaved RB (bottom).

We obtained gate fidelities 99.991318% for 0-th order fit and 95.459883% for 1-st order fit. As expected, gate fidelity is lower when it comes to two-qubit gates.

2) RZZ Gate
The RZZ gate is also known as Ising ZZ coupling gate. It applies an Ising ZZ interaction between two target qubits, creating entanglement by rotating them around the z-axis of the Bloch Sphere by an angle θ. The entanglement is maximal when $\theta = \pi/2$.

	Parameters	$F_{\text{seq}}^{(0)}$	$F_{\text{seq}}^{(1)}$
RB	A	0.700427	0.484393
	B	0.291694	0.508066
	C	-	0.00036
	p	0.99989992	0.99912006
	r	7.506×10^{-5}	$4,3997 \times 10^{-4}$
IRB	A	0.698109	0.486604
	B	0.293991	0.506211
	C	-	0.000768
	$p_{\overline{c}}$	0.99989999	0.9982997
	r_c	7.501×10^{-5}	8.5015×10^{-4}

TABLE VII: Simulation results for Gate RZZ.

The results obtained for gate fidelities were $F_{seq} = \{0.991862, 0.992285, 0.991211, 0.991667, 0.991341\}$, for RB, and $F_{\overline{seq}} = \{0.992122, 0.991374, 0.992090, 0.991341, 0.991341\}$. The graphs below correspond to the data presented in Table VII.

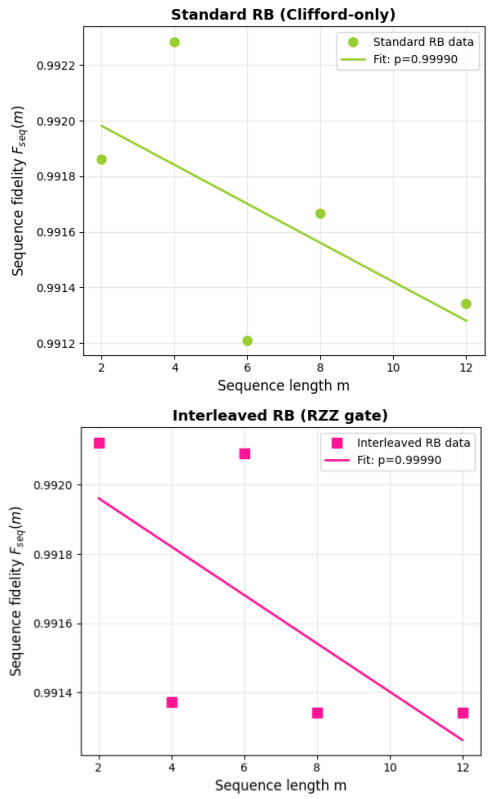

Fig. 6: Fidelity of RZZ via Standard RB (top) and Interleaved RB (bottom).

We obtained gate fidelities 100.00% for 0-th order fit and 99.958946% for 1-st order fit. Similarly to the RZ gate, no real pulses were involved, reducing the noise in the system.

IV. Hardware Implementation

On this section, the RB and IRB protocols are executed on real quantum hardware. Due to the IBM Quantum Platform's maximum job duration of 10 minutes, benchmarking two-qubit gates was not feasible. To enable the evaluation of single-qubit gates within this time constraint, sequence length, number of shots and the number of sequences were reduced to reasonable values ensuring smaller runtime.

A. Gate X

The results for Gate X can be verified in the table below.

	Parameters	$F_{\text{seq}}^{(0)}$	$F_{\text{seq}}^{(1)}$
RB	A	0.698915	0.514625
	B	0.299999	0.498078
	C	-	0.017268
	p	0.99989987	0.96843926
	r	5.006×10^{-5}	1.578037×10^{-2}
IRB	A	0.700488	0.503541
	B	0.299997	0.505838
	C	-	0.009764
	$p_{\overline{c}}$	0.99974177	0.98080762
	r_c	1.2911×10^{-4}	9.59619×10^{-3}

TABLE VIII: Server results for Gate X.

The sequence fidelity values for RB and IRB obtained were, respectively, $F_{seq} = \{0.998000, 0.999333, 0.998571\}$ and $F_{\overline{seq}} = \{1.000000, 1.000000, 0.999333, \}$. The graphs below correspond to the data presented in Table VIII.

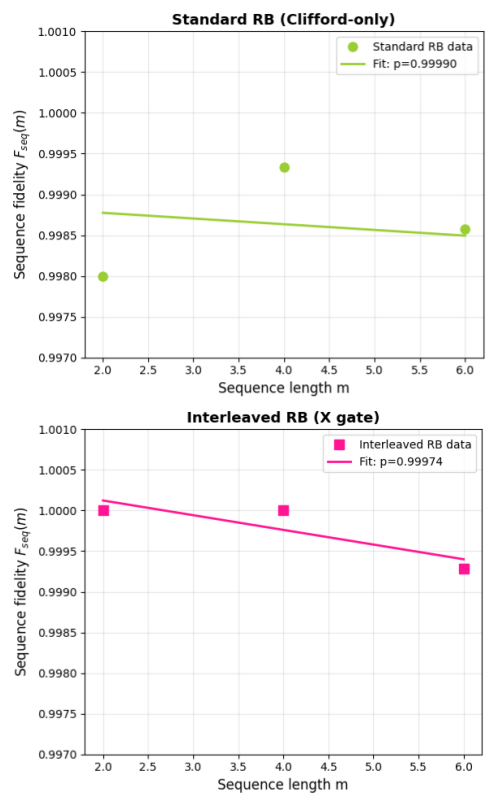

Fig. 7: Real fidelity of X via Standard RB (top) and Interleaved RB (bottom).

Comparing to Table II, we can see that the error for IRB has increased significantly. The format of the IRB graph is due to the fact that for The gate fidelities obtained were both, however, 100.00% overall.

B. Gate SX

The results for Gate SX can be verified in the table below.

	Parameters	$F_{\text{seq}}^{(0)}$	$F_{\text{seq}}^{(1)}$
RB	A	0.700283	0.502471
	B	0.299999	0.502472
	C	-	0.005190
	p	0.99989929	0.98983125
	r	5.036×10^{-5}	5.08438×10^{-3}
IRB	A	0.700283	0.502471
	B	0.299999	0.502472
	C	-	0.005190
	$p_{\overline{c}}$	0.99989929	0.98983125
	r_c	5.036×10^{-5}	5.08438×10^{-3}

TABLE IX: Server results for Gate SX.

The sequence fidelity values for RB and IRB obtained were, respectively, $F_{seq} = \{1.000000, 1.000000, 1.000000\}$ and $F_{\overline{seq}} = \{1.000000, 1.000000, 1.000000\}$. The graphs below correspond to the data presented in Table IX.

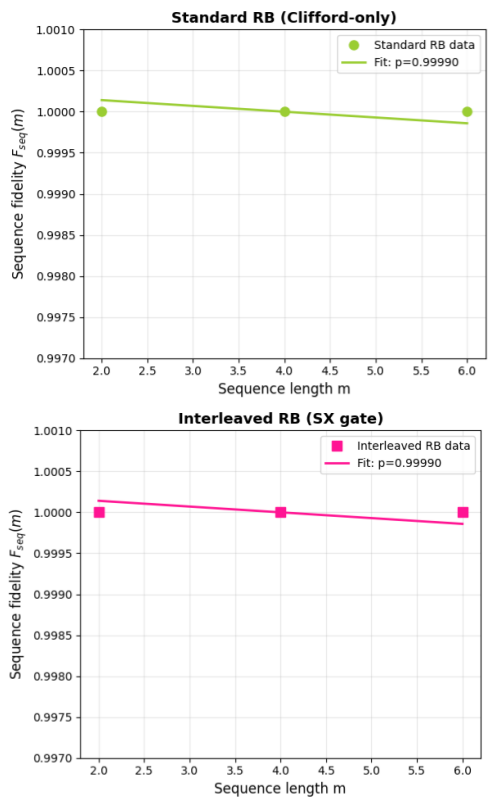

Fig. 8: Real fidelity of SX via Standard RB (top) and Interleaved RB (bottom).

We have obtained Gate Fidelity 99.994964% for 0-th order fit and 100.00% for 1-st order fit. These are quite high values, although the result for the 0-th order fit might be more reasonable.

C. Gate RX

The results for Gate RX can be verified in the table below.

	Parameters	$F_{\text{seq}}^{(0)}$	$F_{\text{seq}}^{(1)}$
RB	A	0.700488	0.503541
	B	0.299997	0.505838
	C	-	0.009764
	p	0.99974177	0.98080762
	r	1.2911×10^{-4}	9.59619×10^{-3}
IRB	A	0.700460	0.504570
	B	0.299997	0.504505
	C	-	0.009434
	$p_{\overline{c}}$	0.99975756	0.98148721
	r_c	1.2122×10^{-4}	9.2564×10^{-3}

TABLE X: Server results for Gate RX.

The results obtained for gate fidelities were $F_{seq} = \{1.000000, 1.000000, 0.999286\}$, for RB, and $F_{\overline{seq}} = \{1.000000, 1.000000, 0.999333\}$. The graphs below correspond to the data presented in Table X.

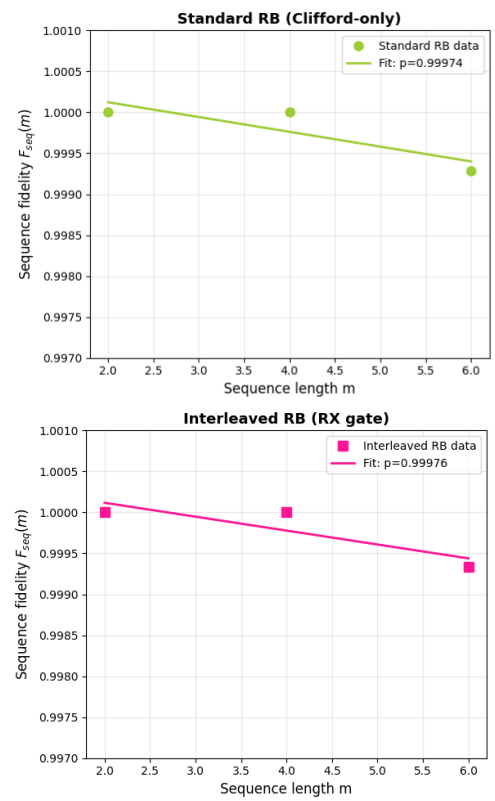

Fig. 9: Real fidelity of RX via Standard RB (top) and Interleaved RB (bottom).

We have obtained Gate Fidelity 100.00 for both 0-th order and 1-st order fit. Similarly to the when using noise simulation, operations around z-axis don't introduce noise to the system, having very high fidelity values.

D. Gate RZ

The results for Gate RZ can be verified in the table below.

	Parameters	$F_{\text{seq}}^{(0)}$	$F_{\text{seq}}^{(1)}$
RB	A	0.700368	0.498358
	B	0.299997	0.502005
	C	-	0
	p	0.99944828	0.99922506
	r	2.7586×10^{-4}	3.8747×10^{-4}
IRB	A	0.700296	0.498259
	B	0.299997	0.502027
	C	-	-0.000002
	$p_{\overline{c}}$	0.99948695	0.99928568
	r_c	2.5653×10^{-4}	3.5716×10^{-4}

TABLE XI: Server results for Gate RZ.

The results obtained for gate fidelities were $F_{seq} = \{1.000000, 0.998000, 0.998462\}$, for RB, and $F_{\overline{seq}} = \{1.000000, 0.998000, 0.998571\}$. The graphs below correspond to the data presented in Table XI.

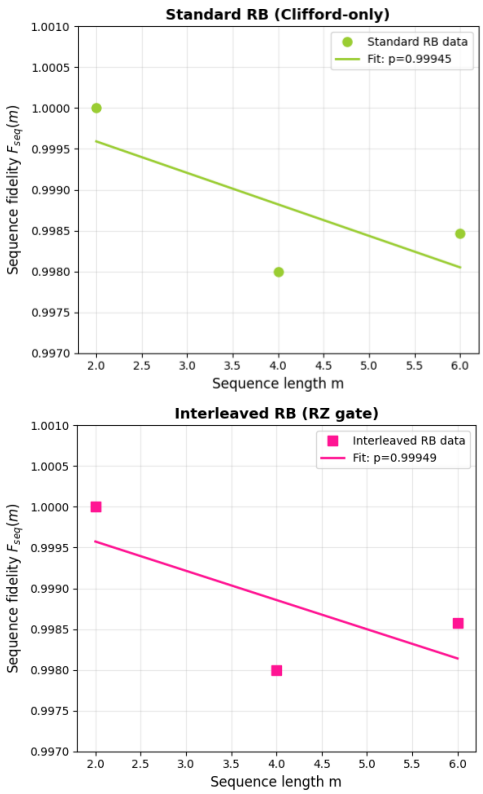

Fig. 10: Real fidelity of RZ via Standard RB (top) and Interleaved RB (bottom).

We have obtained Gate Fidelity 100.00 for both 0-th order and 1-st order fit. Similarly to the results for Gate RZ using noise simulation, operations around z-axis don't introduce noise to the system, having very high fidelity values.

E. Comparison between real and simulated gates

V. Conclusion

The values obtained when running RB and IRB sequences in noise simulated channels are close to the ones when using real hardware, which proves running simulations by use of the primitive **Qiskit Aer** results in values close to those originated from server execution. Although we have obtained extremely high fidelity values, our results have low validity. Working with limited runtime usage raises an imposing constraint, as it is impossible to implement protocols with higher values of sequence length, m, number of sequences, K, and number of shots. To acquire more valid results, it would be necessary to implement these protocols with higher parameters.

References

[1] Atom Computing, *"Highly Scalable Quantum Computing with Neutral Atoms"*, 2023.

[2] IBM, *"The hardware and software for the era of quantum utility is here"*, 2023.

[3] Xanadu, *"Beating classical computers with Borealis"*, 2023.

[4] Quantinuum, *"Quantinuum extends its significant lead in quantum computing, achieving historic milestones for hardware fidelity and Quantum Volume"*, 2024.

[5] SPinQ, *"Scalable Quantum Systems: The Path to Quantum Advantage"*, 2025.

[6] L. H. Pedersen, N. M. Møller, K. Mølmer, *"Fidelity of quantum operations"*, Physics Letters A 367(1-2):47-51, 2007.

[7] Jay M. Gambetta,A. D. Córcoles, S. T. Merkel, B. R. Johnson, John A. Smolin, Jerry M. Chow, Colm A. Ryan, Chad Rigetti, S. Poletto, Thomas A. Ohki, Mark B. Ketchen, M. Steffen1, *"Characterization of addressability by simultaneous randomized benchmarking"*

[8] Forest Benchmarking Documentation, *Randomized Benchmarking (Interleaved)*, , `https://forest-benchmarking.readthedocs.io/en/latest/examples/randomized_benchmarking_interleaved.html`, Accessed: Dec. 15, 2025.
bibitem2qbprotocol A. D. Córcoles, Jay M. Gambetta, Jerry M. Chow, John A. Smolin, Matthew Ware, J. D. Strand, B. L. T. Plourde, M. Steffen1, *"Process verification of two-qubit quantum gates by randomized benchmarking"*

APPENDIX

The most relevant parts of the code used is provided here. In addition, the repository of material can be accessed by `https://github.com/lv-laura/qc_202526`.

Listing 1: Single-qb Clifford Group Generation

```
def normalize_up_to_global_phase(U):
"""Normalize␣unitary␣matrix␣to␣remove␣global␣phase."""
idx = np.argmax(np.abs(U))
phase = np.angle(U.flatten()[idx])
return U * np.exp(-1j * phase)

def generate_single_qubit_cliffords():
   """Generate␣all␣24␣single-qubit␣Clifford␣gates␣using
       ␣H␣and␣S."""
   generating_gates = {'H': lambda qc: qc.h(0), 'S':
       lambda qc: qc.s(0)}
   seen_matrices = []
   circuits = []
   queue = deque([QuantumCircuit(1)]) #We should start
       with identity

   while queue:
      qc = queue.popleft()
      U = Operator(qc).data
      U_norm = normalize_up_to_global_phase(U)

      if any(np.allclose(U_norm, M) for M in
          seen_matrices):
         continue
      seen_matrices.append(U_norm)
      circuits.append(qc)

      for gate_func in generating_gates.values():
         new_qc = qc.copy()
         gate_func(new_qc)
         queue.append(new_qc)

   assert len(circuits) == 24, f"Expected␣24␣Cliffords,
       ␣found␣{len(circuits)}"
   return circuits

#Generate Clifford gates and compute the inverse
cliffords_list = generate_single_qubit_cliffords()
cliffords_matrices = [normalize_up_to_global_phase(
    Operator(qc).data) for qc in cliffords_list]
I2 = np.eye(2, dtype=complex)

cliffords_inverses = {}
for i, U_i in enumerate(cliffords_matrices):
   for j, U_j in enumerate(cliffords_matrices):
      product_norm = normalize_up_to_global_phase(U_i @
          U_j)
      if np.allclose(product_norm, I2):
         cliffords_inverses[i] = j
         break

print(f"Generated␣{len(cliffords_list)}␣Clifford␣
    circuits")
print(f"Computed␣inverse␣mapping␣for␣all␣Cliffords")
```

Function **normalize_up_to_global_phase(U)** takes an unitary matrix and removes the global phase. This way, two matrices that differ only in global phase are considered equal. This is done by finding the element of U with the largest magnitude, extracting its phase, and multiplying U by the complex conjugate of this phase. The result is a matrix which largest element is real and positive.

Function **generate_single_qubit_cliffords()** generates Clifford gates using only H and S gates. A circuit is initialised with the identity matrix, then, a breadth-first search approach is used. A circuit from the queue is popped and its unitary matrix is computed and normalised up to a global phase. If it was corresponds to a previously generated Clifford (was already seen), then it skips to the next. If not, the matrix is added to the list of Clifford circuits. This continues until all unique Clifford matrices are found.

Then, we search which gate in the set is the inverse for each Clifford gate.

Listing 2: Sequence Generation

```
def create_rb_sequence(clifford_circuits,
    cliffords_matrices, m, target_gate_circuit=None,
    seed=None):

rng = np.random.default_rng(seed)
indices = rng.choice(len(clifford_circuits), size=m,
    replace=True)

qc = QuantumCircuit(1, 1) #We only need 1 qb to test
    single qb gates

#Apply random Clifford gates (and optionally interleave
     target gate)
for idx in indices:
   qc.compose(clifford_circuits[idx], inplace=True)
   if target_gate_circuit is not None:
      qc.compose(target_gate_circuit, inplace=True)

#Compute inverse Clifford that returns to |0>
U_fwd = Operator(qc).data
U_fwd_norm = normalize_up_to_global_phase(U_fwd)
I = np.eye(2, dtype=complex)

inverse_idx = None
for j, U_j in enumerate(cliffords_matrices):
   product = normalize_up_to_global_phase(U_fwd_norm @
       U_j)
   if np.allclose(product, I):
      inverse_idx = j
      break

if inverse_idx is None:
   raise RuntimeError("Could␣not␣find␣inverse␣Clifford"
       )

qc.compose(clifford_circuits[inverse_idx], inplace=True
    )
qc.measure(0, 0)
return qc
```

The sequences of length m are defined by choosing gates from the Clifford set at random. This is done with a random number generator initialised with a seed for reproducibility. The single-qubit circuit is initialised with one classical register for later measurement. After the gates are applied, the final inverse gate that returns the system to $|0\rangle$ is calculated similarly to the method above.

Listing 3: Parameter p extraction

```
#Standard RB: No target gate interleaving
sequence_lengths = [2, 4, 6, 8, 12]
num_sequences = 30
shots = 1024
seed_base = 54321

sim = AerSimulator()
survival_probs_rb = [] #To store average survival
    probabilities

print("Running␣Standard␣RB...")
for m in sequence_lengths:
   probs = []
   for k in range(num_sequences):
      seed = seed_base + m * 1000 + k
      rb_seq = create_rb_sequence(
         clifford_circuits=cliffords_list,
         cliffords_matrices=cliffords_matrices,
         m=m,
         target_gate_circuit=None, #No interleaving
         seed=seed
      )
```

```
        rb_seq_transpiled = transpile(rb_seq, backend=
            backend)
        job = sim.run(rb_seq_transpiled, shots=shots,
            noise_model=noise_model)
        result = job.result()
        counts = result.get_counts()
        prob = counts.get('0', 0) / shots
        probs.append(prob)
    avg_prob = np.mean(probs)
    survival_probs_rb.append(avg_prob)
    print(f"  m={m:2d}: F_seq = {avg_prob:.6f}")

print("Standard RB complete")
```

The sequence circuits are simulated and the average probability for each sequence is obtained.

Listing 4: 0th and 1st Order Fit

```
#Zero-order Standard RB Fit: F_seq(m) = A * p^m + B (0
    th order)
m_vals = np.array(sequence_lengths)
F_seq_rb = np.array(survival_probs_rb)

def fit_func(m, A, p, B):
    return A * p**m + B

params_rb, _ = scipy.optimize.curve_fit(
    fit_func, m_vals, F_seq_rb,
    p0=[0.95, 0.98, 0.05],
    bounds=([0.1, 0.85, 0.001], [1, 0.9999, 0.3])
)
A_rb, p_rb, B_rb = params_rb
r_rb = (2 - 1) / 2 * (1 - p_rb) #Average error per
    Clifford gate


___________
#First-order RB fit: F_seq^(1)(m) = A1 * p^m + C1 * (m
    -1) * p^(m-2) + B1

def fit_func_first_order(m, A1, p1, C1, B1):
    m = np.asarray(m, dtype=float)
    return A1 * (p1 ** m) + C1 * (m - 1) * (p1 ** (m -
        2)) + B1

try:
    params_rb_fo, _ = scipy.optimize.curve_fit(
        fit_func_first_order,
        m_vals,
        F_seq_rb,
        p0=[0.5, 0.99, 0.01, 0.5],
        bounds=([0.0, 0.9, -1.0, 0.0], [1.0, 1.0, 1.0,
            1.0])
    )
    A1_rb, p1_rb, C1_rb, B1_rb = params_rb_fo
    r1_rb = (2 - 1) / 2 * (1 - p1_rb)
except Exception as e:
    print(f"First-order RB fit failed: {e}")
```

Parameter p is obtained by use of **`scipy.optimize.curve_fit()`**. For obtaining $p_{\overline{c}}$ in IRB, the same methods are used.

Listing 5: Setting target for IRB

```
#Define target gate (RZ(pi/2) gate)
target_RZ = QuantumCircuit(1)
target_RZ.rz(np.pi/2, 0)
"""
    These two lines can be updated according to the we
    wish to test. For:
        X: target_X = QuantumCircuit(1) , target_X.x(0)
        SX: target_SX = QuantumCircuit(1) , target_SX.sx(0)
        RX: target_RX = QuantumCircuit(1) , target_RX.rx(np.pi
    /2, 0)
        RZ: target_RZ = QuantumCircuit(1) , target_RZ.rz(np.pi
    /2, 0)
    """
#Interleaved RB: RZ(pi/2) gate after each Clifford
survival_probs_irb = []
```

```
print("Running Interleaved RB (RZ(pi/2) gate)...")
for m in sequence_lengths:
    probs = []
    for k in range(num_sequences):
        seed = seed_base + m * 1000 + k + 99999
        irb_seq = create_rb_sequence(
            clifford_circuits=cliffords_list,
            cliffords_matrices=cliffords_matrices,
            m=m,
            target_gate_circuit=target_RZ, # Interleave RZ
                (pi/2) gate
            seed=seed
        )
        irb_seq_transpiled = transpile(irb_seq, backend=
            backend)
        job = sim.run(irb_seq_transpiled, shots=shots,
            noise_model=noise_model)
        result = job.result()
        counts = result.get_counts()
        prob = counts.get('0', 0) / shots
        probs.append(prob)
    avg_prob = np.mean(probs)
    survival_probs_irb.append(avg_prob)
    print(f"  m={m:2d}: F_seq = {avg_prob:.6f}")

print("Interleaved RB complete")
```

The target gate is set to an arbitrary gate and then the IRB sequence is created.

Listing 6: RB sequence for 2-qb

```
    for k in range(num_sequences):
    seed = seed_base + sequence_length * 1000 + k
    rng = np.random.default_rng(seed)
    indices = rng.choice(len(clifford_circuits), size=
        sequence_length, replace=True)
    #Build up the sequence one truncation at a time
    for m in range(1, sequence_length + 1):
        qc = QuantumCircuit(2, 2)
        clifford_product = Clifford(np.eye(4))
        for idx in indices[:m]:
            qc.compose(clifford_circuits[idx], inplace=
                True)
            clifford_product = clifford_product.compose(
                Clifford(clifford_circuits[idx]))
            if target_gate_circuit is not None:
                qc.compose(target_gate_circuit, inplace=
                    True)
                clifford_product = clifford_product.compose
                    (Clifford(target_gate_circuit))
        # Compute inverse for this truncation
        inv_cliff = clifford_product.adjoint()
        qc.compose(inv_cliff.to_circuit(), inplace=True)
        qc.measure([0, 1], [0, 1])
        transpiled = transpile(qc, backend=backend)
        job = sim.run(transpiled, shots=shots,
            noise_model=noise_model)
        result = job.result()
        counts = result.get_counts()
        prob = counts.get('00', 0) / shots # For 2 qubits
        survival_probs[m-1, k] = prob
#Average over sequences for each truncation length
avg_survival_probs = np.mean(survival_probs, axis=1)
return avg_survival_probs
```

The truncations are defined.